AI Meets Formal Verification: Practice and Prospects

Subtitle

Liao Zhang

Beneficial AI Foundation

November 5, 2026

Overview

1. Background: Beneficial AI Foundation and Signal Shot
2. Practice
3. Prospects

Beneficial AI Foundation

Mission

Keep AI beneficial for humanity through technical AI safety research.

- Founded by Max Tegmark in 2022.
- Supports research through an in-house team, fellowships, and university partnerships.
- Research interests include red-teaming, interpretability, and guaranteed safe AI.
- For this talk, the focus is **AI-assisted formal verification**: using AI to help build software with machine-checked guarantees.

Connection to practice: cryptographic software provides concrete, demanding verification tasks.

Sources: Beneficial AI Foundation; Signal Shot.

Signal Shot: A Shared Verification Challenge

Launched in April 2026: a collaborative initiative to formally verify the Signal protocol and application using Lean.

- **Technical ambition:** work toward machine-checked guarantees across the protocol stack and its implementation.
- **Engineering ambition:** develop reusable tools for protocol design and continuous verification as software changes.
- **Community ambition:** bring together Lean, cryptography, software verification, and AI-assisted theorem proving.
- **Concrete workstreams:** the initiative links verification repositories for libsignal, PQXDH, and SPQR.

From the Initiative to This Talk

SPQR is our practical case study: writing specifications, constructing Lean proofs, and understanding where AI helps and where human judgment remains essential.

Source: Signal Shot: objectives and project repositories.

SPQR: What It Does and What We Verify

- **What is it?** SPQR (Sparse Post-Quantum Ratchet) is Signal's Rust implementation of a post-quantum ratchet for secure messaging.
- **What does it do?** It uses incremental ML-KEM-768 exchanges to generate fresh shared secrets for updating messaging keys.
- **Why sparse?** Key-exchange data is carried in chunks across messages; a new shared secret need not be produced for every message.

Our Verification Project

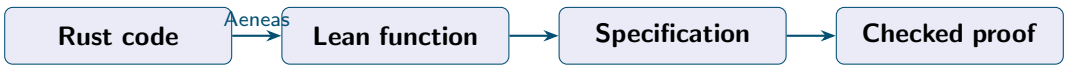
Use Aeneas and Lean to establish functional properties of the Rust implementation: chunk encoding and decoding, message serialization, ratchet state transitions, and epoch management.

Example guarantee: along one participant's successful state-machine run, at most one key is emitted per epoch, and emitted epochs strictly increase.

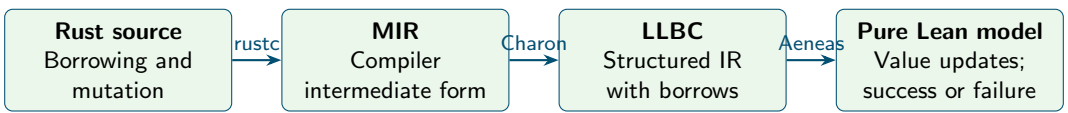
These implementation-level results depend on stated assumptions and external models; they do not constitute a complete cryptographic security proof.

SPQR: From Rust Code to Checked Proofs

Target: functional correctness of Signal's Sparse Post-Quantum Ratchet.



Inside the translation: Rust control flow becomes a functional model



- **Extraction:** Charon simplifies MIR to ULLBC, then reconstructs loops and branches into LLBC (Low-Level Borrow Calculus).
- **Translation:** Aeneas symbolically executes LLBC, tracking values, borrows, and loans. Mutation becomes explicit value updates in pure functions.
- **Semantic foundation:** formal results relate the borrow-based semantics to a heap-and-address model for the formalized language, not the entire Rust toolchain.

Sources: Aeneas / Charon documentation; Functional Translation (2022); Symbolic Semantics (2024).

SPQR Example: A Byte Comparison Loop

Compare two byte slices by accumulating differences with bitwise XOR and OR.

Rust: Mutation and a `for` Loop

```
fn compare(lhs: &[u8], rhs: &[u8]) -> u8 {
    let mut r: u8 = 0;
    for i in 0..lhs.len() {
        r |= lhs[i] ^ rhs[i];
    }
    is_non_zero(r)
}
```

`is_non_zero(r)` returns 0 if $r = 0$, and 1 otherwise.

Lean: Explicit State and `Result`

```
def compare (lhs rhs : Slice U8) : Result U8 := do
  let n := Slice.len lhs
  let r ← compare_loop
    { start := 0#usize, «end» := n } lhs rhs 0#u8
  is_non_zero r
```

Loop body at index `i` (names shortened):

```
let a ← Slice.index_usize lhs i
let b ← Slice.index_usize rhs i
let d ← lift (a ^^^ b)
let r1 ← lift (r ||| d)
ok (cont (iter1, r1))
```

Rust construct	**Lean representation**
`&[u8]`	`Slice U8`: a borrowed byte slice becomes a value.
`let mut r; for`	`loop` threads state `(iter, r)`; `cont` continues, `done` exits.
`lhs[i]`	`Slice.index_usize`: a checked access in `Result`; failure propagates.

SPQR Example: Specification and Proof

Specification and Top-Level Proof (compare_spec)

```
theorem compare_spec (lhs rhs : Slice U8)
    (hlen : lhs.length = rhs.length) :
    compare lhs rhs {{ (result : U8) ⇒
      result.val =
        if (∀ (j : Nat), j < lhs.length → lhs.val[j]! = rhs.val[j]!)
        then 0 else 1 }} := by
  unfold compare
  step*
  grind
```

Meaning: equal-length inputs ⇒ successful execution; return 0 for equal slices and 1 otherwise.

Loop invariant: $r = 0 \iff \forall j < i,\ lhs[j] = rhs[j]$.

Initialize $i = 0$, $r = 0$: the empty prefixes match.

Preserve OR accumulates XOR differences; bit-vector lemmas establish the invariant at $i + 1$.

Finish $n - i$ decreases; equal lengths ensure valid accesses. At $i = n$, use `is_non_zero`.

`step*` applies helper specifications; `grind` closes the logical goal. The theorem establishes functional behavior, not constant-time execution.

Dalek: What It Does and What We Verify

- **What is it?** `curve25519-dalek` is a Rust library for group operations on Curve25519 (Edwards and Montgomery forms) and Ristretto.
- **What does it compute?** Point addition $P + Q$, scalar multiplication $[s]P$, and conversions between points, scalars, and bytes.
- **Where is it used?** These operations support key agreement, digital signatures, and zero-knowledge proofs; protocol libraries build on them.

Our Verification Project

Use Aeneas and Lean to verify selected functions in the 64-bit serial backend: field arithmetic modulo $2^{255} - 19$, byte encodings, and curve operations.

Example guarantee: for a valid point P and scalar bytes with the top bit clear, scalar multiplication succeeds and returns a valid point Q with

$$\text{toPoint}(Q) = [\text{decode}(s)]\,\text{toPoint}(P).$$

This connects the concrete Rust computation to the mathematical group operation.

Lessons from Practice

- AI capabilities in constructing Lean proofs have improved substantially.
- Simple prompts appear sufficient in most cases.
- The clarity of AI-generated explanations has become a key bottleneck.
- A simple design is sufficient for the agent harness.
- Cross-model review is crucial when writing specifications.

Advances in AI-Assisted Lean Proofs

- One year ago, AI struggled to construct challenging proofs.
- At that time, proofs involving loop invariants were beyond its capabilities in my experience.
- In my recent work, I have not encountered a proof that AI could not complete.

Simple Prompts Often Suffice

- One year ago, Claude Code skills were developed to guide AI workflows.
- These skills use Markdown documents to provide instructions to AI agents.
- The Model Context Protocol (MCP) provides an additional mechanism for tool integration.
- Examples include Lean 4 skills and Lean MCP integrations.

Simple Prompts Often Suffice

- Current AI systems appear substantially more capable.
- I examined how Claude Code operates in my Lean workflow.
- In the sessions I examined, it used neither skills nor MCP tools to construct Lean proofs.
- Why might MCP tools be unnecessary in this workflow?
- MCP tools can expose the tree structure of proofs.
- However, this view may omit global information, such as the number of remaining `sorry` placeholders.
- Given current AI capabilities in Lean, elaborate prompts appear to offer limited additional benefit.
- The most useful instruction appears to be: "Decompose the proof and compile after each completed step."
- Claude Code skills appear less useful in this workflow than they once were.

Clarity of AI-Generated Explanations

- Despite its capabilities, AI still requires human guidance.
- Clear, comprehensible explanations are essential for effective human oversight.
- AI systems tend to produce lengthy responses.
- My most frequent instruction is: "Please shorten your response while preserving precision and clarity."
- AI models should communicate their results in language that users can readily understand.

A Simple Harness Design Is Sufficient

- Use a simple workflow to coordinate proof development.
- Assign a central agent to determine which proof node to expand next.
- Determine which proof nodes can be addressed within the current iteration.
- Record progress after each iteration.
- Reset the working context periodically to prevent excessive accumulation.

A Simple Harness Design Is Sufficient

- Initially, I was unable to complete the proof.
- The `to_bytes` implementation exceeds 700 lines of code, and proof attempts frequently time out.
- The only effective workflow I found was to decompose complex theorems into simpler lemmas.
- Compile after each incremental improvement.

Cross-Model Review for Specification Writing

- AI can generate specifications, but these specifications may contain errors.
- Complement manual review with independent reviews by different AI models.
- This approach is particularly useful for less critical specifications.
- Examples include helper lemmas and functions outside the public API.
- Cross-model review is a practical approach to checking these specifications.

Prospects: Important Directions of AI for formal verification

- Efficient Inference
- Understandable Formal verification

Prospects: Important Directions of AI for formal verification

- Efficient Inference
- Understandable Formal verification

Specification as a Central Research Challenge

- As proof construction becomes easier, selecting the right statements may require more attention.
- Generate candidate specifications from documentation, interfaces, and intended use cases.
- Make assumptions, boundary conditions, and excluded behaviors explicit.
- Require human review of whether each formal statement captures the intended requirement.

Evaluating the Strength of Specifications

- Check whether specifications rule out plausible implementation errors.
- Use examples and counterexamples to clarify the behaviors a specification permits.
- Investigate mutation-based evaluation: would an incorrect implementation still satisfy the statement?
- Examine whether assumptions are consistent and whether conclusions provide meaningful guarantees.

Scaling through Compositional Verification

- Decompose large verification tasks into components with explicit contracts.
- Develop reusable lemmas for recurring patterns, including loops and data conversions.
- Track dependencies between local guarantees and system-level properties.
- Use projects such as Dalek and SPQR to explore how these workflows scale.

More Effective Agent Coordination

- Maintain a shared record of proof obligations, dependencies, and completed results.
- Assign bounded tasks with clear completion criteria to individual agents.
- Separate specification review from proof construction to encourage independent scrutiny.
- Evaluate whether additional agents justify their coordination and computational costs.

Human Oversight through Clear Explanations

- Present each result with its formal statement, assumptions, and a concise explanation.
- Explain how the statement corresponds to the original software requirement.
- Highlight unresolved obligations and decisions that require human judgment.
- Treat clarity and review effort as explicit evaluation criteria.

Verification throughout Software Maintenance

- Integrate proof checking into the development and review workflow.
- Identify which specifications and proofs are affected by implementation changes.
- Explore AI-assisted proof repair while preserving the intended guarantees.
- Review changes to theorem statements and assumptions as carefully as changes to code.

Evaluating Practical Progress

- Measure completion rates on representative verification tasks.
- Report computational cost, elapsed time, and human intervention.
- Assess specification quality, proof maintainability, and reproducibility.
- Track remaining proof gaps and assumptions alongside successful results.
- Compare simple workflows with more elaborate agent architectures.

A Proposed Roadmap

1. Establish reproducible workflows for bounded verification tasks.
2. Strengthen specification review and build reusable proof libraries.
3. Extend verification across component boundaries and software revisions.
4. Evaluate sustained use in realistic development settings.

Long-Term Objective

Make rigorous, understandable, and maintainable verification accessible to more developers.

The End